Real-world (conflicting requirements design) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conflicting_requirements_design.md`: real requests come from several different people who want genuinely different things from the same system. This exercise trains naming those conflicts *explicitly, in writing*, before any design work starts, and turning what would otherwise be a hidden decision into a written, visible one the stakeholders can actually see and correct. **If not:** your code would quietly pick a winner between stakeholders on its own, and nobody would even know a real decision had been made.

Write your own design first, in `practice/conflicting_requirements_design.md`.

This solution uses the README's solo practice prompt (live sessions bring a fresh one): three stakeholders on the ContentForge-style content pipeline from the Intermediate exercise. **Marketing** wants every article published automatically within minutes, for speed. **Legal** wants a human to review every article before publish, no exceptions. **Finance** wants API/compute cost capped. Nobody has said what should happen when Marketing's speed and Legal's review both apply to the same article.

Solution

Story: this lists each conflict in a plain table first, then proposes one concrete architectural pattern (tiered-risk routing) that genuinely serves both sides, plus a separate, independent cost cap — with the exact risk cutoff clearly marked as an *assumption* for the real stakeholders to confirm, not a fact. **If not:** the risk-tier line would quietly become your own private guess, shipped straight into production as unstated company policy.

Start by writing the conflicts down, plainly, before any architecture:

Requirement	From	Conflicts with
Publish within minutes, fully autonomous	Marketing	Legal's "review everything"
Human reviews every article, no exceptions	Legal	Marketing's speed target

Requirement	From	Conflicts with
Cap API/compute cost	Finance	Nothing directly, but unaddressed by either naive design above

Then propose a **tiered-risk conditional routing** design (Doc11's routing family) instead of picking a side:

- **Risk score** computed per article from: topic sensitivity (does it touch legal/financial/medical claims?), source confidence (how strong was the research phase's sourcing?), fact-check result (did it pass clean, or with caveats?).
- **Low risk** (routine topic, strong sources, clean fact-check pass) → **auto-publish**, serving Marketing's speed requirement for the large majority of ordinary content.
- **High risk** (sensitive topic, weak sourcing, or fact-check caveats) → **routes to human review**, serving Legal's actual concern — which is very likely about liability exposure on specific kinds of claims, not about every single routine article.
- **Cost cap**, addressed as its own explicit guardrail: a per-article and daily spend ceiling on the research/fact-check API calls, with the pipeline pausing and alerting (not silently degrading) if it's hit — this is Finance's requirement, and it doesn't get resolved by the risk-tier decision at all, so it needs its own named piece.

Critically, this design does **not** present the risk-tier cutoff as a fact — it's flagged explicitly: *"Proposed: auto-publish only when topic sensitivity is low AND fact-check passes with no caveats. Everything else requires human review. This line is a starting proposal, not a decision — Legal and Marketing need to agree on exactly where it sits, and who has authority to move it later."*

Why each piece is here, not added for its own sake: The naive designs — autopublish everything, or review everything — are actually *easier* to build than the tiered design, which is exactly why they're tempting and exactly why they're wrong. Each one resolves a real disagreement between two stakeholders by simply implementing one side's request and never surfacing that a choice was made at all. That's not neutral — it's a hidden policy decision, made by whoever wrote the code, about something (legal risk tolerance, or how much speed the business actually needs) that isn't the engineer's call to make silently. Most of the tiered design's components (the risk score, the two routing branches, the cost cap) map directly to a specific named conflict or an unaddressed requirement. What makes it the right answer is that it turns an implicit, buried decision into an explicit, written one that the people who own that decision can see and correct.

Presenting this in an interview: Open by naming the conflicts plainly — that's the single most important move in this exercise, and skipping straight to a design (even a good one) without first saying "here's where these requirements actually disagree" reads as having missed the point of a real-world, messy prompt. Then present the tiered-risk design as your *proposed* resolution, explicitly flagged as resting on an assumption, with the specific question you'd take back to Legal and Marketing named out loud. Being able to say "here's my best guess, here's exactly what I'm

assuming, and here's the one question I'd need answered to firm this up" is the actual senior-level answer — not a design that never admits a guess was made at all.